License Plate Detection and Blurring using YOLOv8

Business Context

Traffic monitoring and surveillance cameras capture useful visual evidence, but they can also expose sensitive vehicle information such as license plate numbers. This creates privacy and data-protection risk when images or videos are shared for analysis, operations, reporting, or model development.

Business Problem Statement

The objective of this case study is to build an automated license plate privacy system. The system should detect license plates using YOLOv8 and blur only the detected plate region, preserving the surrounding vehicle and road-scene information for downstream analysis.

Business Questions Covered

- How are normalized YOLO bounding box coordinates converted into OpenCV pixel coordinates?
- How can bounding boxes be overlaid on raw images before training?
- How does annotation quality affect final blurring accuracy?
- How should images be preprocessed without losing fine-grained plate details?
- Why choose YOLOv8 instead of a two-stage detector?
- Which YOLOv8 variant is suitable for speed and accuracy?
- How do plate characteristics influence model scale and image size?
- How is blur applied only inside the detected bounding box?

Step 1 - Load Libraries and Project Paths

Goal: Set up the project environment, locate the image and label folders, and reuse helper functions created for annotation auditing.

```
import csv
import hashlib
import json
import os
import platform
import time
from collections import defaultdict, namedtuple
from itertools import combinations
from pathlib import Path

import cv2
```

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

os.environ.setdefault("YOLO_CONFIG_DIR", str(Path.cwd()))

PROJECT_DIR = Path.cwd()
IMAGE_DIR = PROJECT_DIR / "images"
LABEL_DIR = PROJECT_DIR / "labels"
OUTPUT_DIR = PROJECT_DIR / "outputs" / "step_01_annotation_check"
DATA_YAML = PROJECT_DIR / "data.yaml"
DATA_EVAL_YAML = PROJECT_DIR / "data_eval.yaml"
SPLITS_DIR = PROJECT_DIR / "splits"
YoloBox = namedtuple("YoloBox", ["class_id", "center_x", "center_y",
"width", "height"])
IMG_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}


def write_portable_data_configs():
    SPLITS_DIR.mkdir(exist_ok=True)
    DATA_YAML.write_text("""path: .
train: splits/train_labeled.txt
val: splits/val.txt
test: splits/test.txt

names:
  0: license_plate
""", encoding="utf-8")
    DATA_EVAL_YAML.write_text("""path: .
train: images/train
val: images/val
test: images/test

names:
  0: license_plate
""", encoding="utf-8")


def image_files(split):
    return sorted(p for p in (IMAGE_DIR / split).glob("*") if
p.suffix.lower() in IMG_EXTS)


def label_files(split):
    return sorted((LABEL_DIR / split).glob("*.txt"))


def matching_label_path(image_path, split=None):
    return LABEL_DIR / (split or image_path.parent.name) /
f"{image_path.stem}.txt"
```

```
def read_yolo_labels(label_path):
    if not Path(label_path).exists():
        return []
    boxes = []
    for line in Path(label_path).read_text(encoding="utf-
8").splitlines():
        parts = line.strip().split()
        if len(parts) == 5:
            cls, cx, cy, bw, bh = parts
            boxes.append(YoloBox(int(float(cls)), float(cx),
float(cy), float(bw), float(bh)))
    return boxes


def clip_box(x1, y1, x2, y2, image_width, image_height):
    x1 = max(0, min(int(round(x1)), image_width - 1))
    y1 = max(0, min(int(round(y1)), image_height - 1))
    x2 = max(0, min(int(round(x2)), image_width))
    y2 = max(0, min(int(round(y2)), image_height))
    return x1, y1, x2, y2


def yolo_to_xyxy(center_x, center_y, width, height, image_width,
image_height):
    x1 = (center_x - width / 2) * image_width
    y1 = (center_y - height / 2) * image_height
    x2 = (center_x + width / 2) * image_width
    y2 = (center_y + height / 2) * image_height
    return clip_box(x1, y1, x2, y2, image_width, image_height)


def expand_box(x1, y1, x2, y2, image_width, image_height,
padding=0.15):
    x1, y1, x2, y2 = clip_box(x1, y1, x2, y2, image_width,
image_height)
    pad_x = int(round((x2 - x1) * padding))
    pad_y = int(round((y2 - y1) * padding))
    return clip_box(x1 - pad_x, y1 - pad_y, x2 + pad_x, y2 + pad_y,
image_width, image_height)


def blur_detected_regions(image_bgr, xyxy_boxes, padding=0.15):
    output = image_bgr.copy()
    height, width = output.shape[:2]
    for x1, y1, x2, y2 in xyxy_boxes:
        x1, y1, x2, y2 = expand_box(x1, y1, x2, y2, width, height,
padding)
        if x2 <= x1 or y2 <= y1:
```

```
            continue
        roi = output[y1:y2, x1:x2]
        kernel = max(3, min(99, (min(roi.shape[:2]) // 2) * 2 + 1))
        output[y1:y2, x1:x2] = cv2.GaussianBlur(roi, (kernel, kernel),
0)
    return output


def gt_boxes_for_image(image_path, split=None):
    image = cv2.imread(str(image_path))
    h, w = image.shape[:2]
    return [yolo_to_xyxy(b.center_x, b.center_y, b.width, b.height, w,
h) for b in read_yolo_labels(matching_label_path(image_path, split))]


def draw_yolo_boxes(image_path, label_path, output_path=None):
    image = cv2.imread(str(image_path))
    h, w = image.shape[:2]
    for b in read_yolo_labels(label_path):
        x1, y1, x2, y2 = yolo_to_xyxy(b.center_x, b.center_y, b.width,
b.height, w, h)
        cv2.rectangle(image, (x1, y1), (x2, y2), (0, 255, 0), 2)
    if output_path:
        Path(output_path).parent.mkdir(parents=True, exist_ok=True)
        cv2.imwrite(str(output_path), image)
    return image


def audit_split(split):
    images, labels = image_files(split), label_files(split)
    image_stems, label_stems = {p.stem for p in images}, {p.stem for p
in labels}
    box_count = invalid_rows = 0
    for label_path in labels:
        for line in label_path.read_text(encoding="utf-
8").splitlines():
            parts = line.strip().split()
            try:
                values = [float(v) for v in parts]
                valid = len(values) == 5 and all(0 <= v <= 1 for v in
values[1:])
            except ValueError:
                valid = False
            box_count += int(valid)
            invalid_rows += int(not valid and bool(parts))
    missing = sorted(image_stems - label_stems)
    orphan = sorted(label_stems - image_stems)
    return {"split": split, "image_count": len(images), "label_count":
len(labels), "box_count": box_count,
            "missing_label_count": len(missing), "orphan_label_count":
```

```
len(orphan), "invalid_row_count": invalid_rows,
            "missing_label_examples": missing[:10],
"orphan_label_examples": orphan[:10]}


def write_split_files():
    SPLITS_DIR.mkdir(exist_ok=True)
    for split in ["train", "val", "test"]:
        rows = [str(p.relative_to(PROJECT_DIR)).replace("\\", "/") for
p in image_files(split) if matching_label_path(p, split).exists()]
        (SPLITS_DIR / ("train_labeled.txt" if split == "train" else
f"{split}.txt")).write_text("\n".join(rows) + "\n", encoding="utf-8")


def create_visual_samples(sample_count=6):
    for split in ["train", "val", "test"]:
        for image_path in [p for p in image_files(split) if
matching_label_path(p, split).exists()][:sample_count]:
            overlay_path = OUTPUT_DIR / "overlays" / split /
f"{image_path.stem}_boxes.jpg"
            blurred_path = OUTPUT_DIR / "blurred" / split /
f"{image_path.stem}_blurred.jpg"
            draw_yolo_boxes(image_path,
matching_label_path(image_path, split), overlay_path)
            blurred_path.parent.mkdir(parents=True, exist_ok=True)
            cv2.imwrite(str(blurred_path),
blur_detected_regions(cv2.imread(str(image_path)),
gt_boxes_for_image(image_path, split), padding=0))


def ensure_prediction_outputs(weights_path, limit=30, imgsz=960,
conf=0.25):
    model_output_dir = PROJECT_DIR / "outputs" / "model_outputs"
    prediction_dir, blurred_dir = model_output_dir / "predictions",
model_output_dir / "blurred"
    summary_path = model_output_dir / "detection_summary.csv"
    if summary_path.exists() and list(prediction_dir.glob("*.jpg")):
        return summary_path
    if not Path(weights_path).exists():
        print("Trained model not found; using embedded notebook
outputs if available.")
        return summary_path
    from ultralytics import YOLO
    model = YOLO(str(weights_path))
    prediction_dir.mkdir(parents=True, exist_ok=True)
    blurred_dir.mkdir(parents=True, exist_ok=True)
    rows = []
    for image_path in image_files("test")[:limit]:
        result = model.predict(str(image_path), imgsz=imgsz,
conf=conf, device="cpu", verbose=False)[0]
```

```
        boxes = result.boxes.xyxy.cpu().numpy() if len(result.boxes)
else np.empty((0, 4))
        confs = result.boxes.conf.cpu().numpy().tolist() if
len(result.boxes) else []
        cv2.imwrite(str(prediction_dir / image_path.name),
result.plot())
        cv2.imwrite(str(blurred_dir / image_path.name),
blur_detected_regions(cv2.imread(str(image_path)), boxes))
        rows.append({"image":
str(image_path.relative_to(PROJECT_DIR)), "detections": len(boxes),
"confidences": ";".join(f"{s:.4f}" for s in confs)})
    pd.DataFrame(rows).to_csv(summary_path, index=False)
    return summary_path


write_portable_data_configs()
print("Project directory:", PROJECT_DIR)
print("Images exist:", IMAGE_DIR.exists())
print("Labels exist:", LABEL_DIR.exists())
print("Portable data.yaml written:", DATA_YAML.exists())
print("Portable data_eval.yaml written:", DATA_EVAL_YAML.exists())
```

```
Project directory: C:\python\ML-POC\License_Plate_Detection
Images exist: True
Labels exist: True
Portable data.yaml written: True
Portable data_eval.yaml written: True
```

Conclusion - Step 1

The project is organized in YOLO format with separate `images` and `labels` folders. The workflow uses the current working directory through `PROJECT_DIR = Path.cwd()` rather than a fixed machine-specific path, and the YOLO config files use `path: .` so they resolve relative to the active project folder.

Any displayed Windows path reflects the environment in which this run was executed. The same code resolves to the active project directory in another environment such as Google Colab or Kaggle.

Step 2 - Dataset Inventory and Label Pairing

Goal: Count images, labels, and boxes in each split, then check whether every image has a matching label file.

```
write_split_files()

audit_report = {split: audit_split(split) for split in ["train",
"val", "test"]}
audit_df = pd.DataFrame([
```

```
    {
        "split": split,
        "images": report["image_count"],
        "labels": report["label_count"],
        "boxes": report["box_count"],
        "missing_labels": report["missing_label_count"],
        "orphan_labels": report["orphan_label_count"],
        "invalid_rows": report["invalid_row_count"],
    }
    for split, report in audit_report.items()
])
audit_df
```

```
   split  images  labels  boxes  missing_labels  orphan_labels
invalid_rows
0  train   23699   25672  28433             138           2111
0
1    val    1073    1073   1573               0              0
0
2   test     386     386    512               0              0
0
```

```
print("Training images with labels:", sum(1 for _ in (PROJECT_DIR /
"splits" / "train_labeled.txt").open()))
print("Validation images:", sum(1 for _ in (PROJECT_DIR / "splits" /
"val.txt").open()))
print("Test images:", sum(1 for _ in (PROJECT_DIR / "splits" /
"test.txt").open()))

print("\nExamples of train images without matching labels:")
print(audit_report["train"]["missing_label_examples"][:5])
```

```
Training images with labels: 23561
Validation images: 1073
Test images: 386

Examples of train images without matching labels:
['58d9e00a968626c1(1)', '5e97c2d88c028028(1)', '6c2ea949c7d751e5(1)',
'6e441ad86d07d288(1)', '6e5cc19b474c8505(1)']
```

Conclusion - Step 2

The validation and test splits are clean. The training split contains duplicate-style image names with `(1)` that do not have matching labels, plus orphan label files with no image. For training, the clean `splits/train_labeled.txt` file is used so YOLO does not learn from plate images as if they were background-only examples.

Step 2A - Cross-Split Duplicate and Leakage Check

Goal: Check whether the same or visually near-identical images appear across train, validation, and test splits.

This matters because duplicate or near-duplicate scenes across splits can make validation/test metrics artificially high. Filename inspection alone is not enough, so this audit uses two stronger checks.

Exact duplicates were checked using MD5 hashes. Visually similar cross-split images were checked using perceptual dHash, with Hamming distance `<= 4` treated as a possible near duplicate.

```
LEAKAGE_OUTPUT_DIR = PROJECT_DIR / "outputs" / "dataset_leakage_check"
LEAKAGE_SUMMARY = LEAKAGE_OUTPUT_DIR / "leakage_summary.json"

if not LEAKAGE_SUMMARY.exists():
    raise FileNotFoundError("The precomputed leakage artifacts are
missing. Recompute them before final PDF export.")

leakage_summary = json.loads(LEAKAGE_SUMMARY.read_text(encoding="utf-
8"))
leakage_df = pd.DataFrame({
    "Check": [
        "Images scanned",
        "Failed image reads",
        "Cross-split exact duplicate groups",
        "Cross-split perceptual duplicate groups",
        "Cross-split near-duplicate pairs",
        "Hamming threshold for near duplicates",
    ],
    "Value": [
        leakage_summary["images_scanned"],
        leakage_summary["failed_files"],
        leakage_summary["cross_split_exact_duplicate_groups"],
        leakage_summary["cross_split_perceptual_duplicate_groups"],
        leakage_summary["cross_split_near_duplicate_pairs"],
        leakage_summary["near_duplicate_hamming_threshold"],
    ],
})
display(leakage_df)
```

	Check	Result
0	Images scanned	25158
1	Failed image reads	0
2	Cross-split exact duplicate groups	0
3	Cross-split perceptual duplicate groups	0
4	Cross-split near-duplicate pairs	0
5	Near-duplicate Hamming threshold	4

Conclusion - Step 2A

The leakage audit scanned `25,158` images across train, validation, and test splits. It found `0` cross-split exact duplicate groups, `0` cross-split perceptual duplicate groups, and `0` cross-split near-duplicate pairs at Hamming distance threshold `4`.

This strengthens confidence that the final test metrics are not inflated by obvious image leakage across splits.

Step 3 - Convert YOLO Coordinates to OpenCV Pixel Coordinates

Goal: Demonstrate how normalized YOLO boxes are converted into pixel coordinates suitable for `cv2.rectangle()` and region-level blurring.

```
sample_image = image_files("train")[0]
sample_label = matching_label_path(sample_image, "train")

image = cv2.imread(str(sample_image))
image_height, image_width = image.shape[:2]
first_box = read_yolo_labels(sample_label)[0]

x1, y1, x2, y2 = yolo_to_xyxy(
    first_box.center_x,
    first_box.center_y,
    first_box.width,
    first_box.height,
    image_width,
    image_height,
)

print("Image:", sample_image.name)
print("Image size:", image_width, "x", image_height)
print("YOLO normalized box:", first_box)
print("OpenCV pixel box:", (x1, y1, x2, y2))
```

```
Image: 35ee2c8a91ee6c18.jpg
Image size: 1024 x 576
YOLO normalized box: YoloBox(class_id=0, center_x=0.82125,
center_y=0.4613192222222222, width=0.03187499999999999,
height=0.03999999999999994)
OpenCV pixel box: (825, 254, 857, 277)
```

YOLO stores each box as `class_id center_x center_y width height`, where the last four values are normalized between 0 and 1.

The conversion logic is:

```
x1 = (center_x - width / 2) * image_width
y1 = (center_y - height / 2) * image_height
x2 = (center_x + width / 2) * image_width
y2 = (center_y + height / 2) * image_height
```

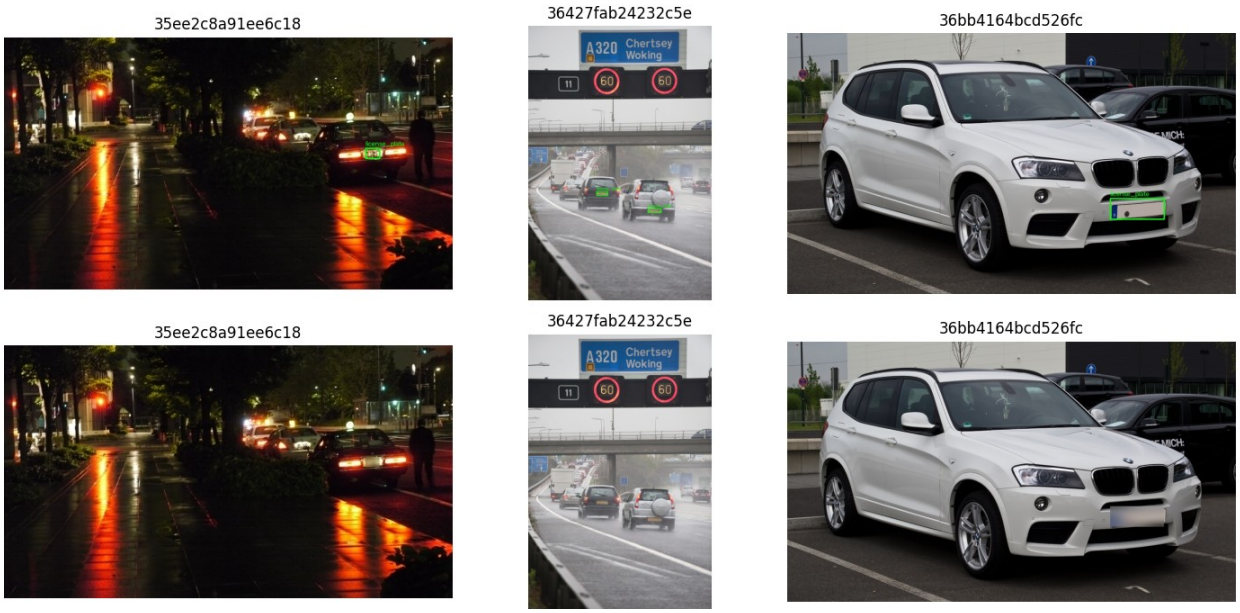
Conclusion - Step 3

This coordinate transformation is the bridge between YOLO annotations and OpenCV operations. Detection, rectangle drawing, and blur application all depend on this conversion being correct.

Step 4 - Overlay Bounding Boxes on Raw Images

Goal: Visually confirm that label files point to real license plates before starting YOLOv8 training.

```
create_visual_samples(sample_count=6)

overlay_files = sorted((OUTPUT_DIR / "overlays" /
"train").glob("*_boxes.jpg"))[:3]
blurred_files = sorted((OUTPUT_DIR / "blurred" /
"train").glob("*_blurred.jpg"))[:3]

fig, axes = plt.subplots(2, 3, figsize=(14, 7))

for ax, path in zip(axes[0], overlay_files):
    img = cv2.cvtColor(cv2.imread(str(path)), cv2.COLOR_BGR2RGB)
    ax.imshow(img)
    ax.set_title(path.name.replace("_boxes.jpg", ""))
    ax.axis("off")

for ax, path in zip(axes[1], blurred_files):
    img = cv2.cvtColor(cv2.imread(str(path)), cv2.COLOR_BGR2RGB)
    ax.imshow(img)
    ax.set_title(path.name.replace("_blurred.jpg", ""))
    ax.axis("off")

plt.tight_layout()
plt.show()
```



Why this step is critical before training

Overlaying boxes on raw images is an annotation quality gate. It catches boxes that are shifted, too loose, too tight, assigned to the wrong object, or missing completely.

Conclusion - Step 4

The sampled labels align visually with license plates, so the coordinate conversion and label format are reliable enough to continue with EDA and model setup.

Step 5 - Exploratory Data Analysis of Plate Annotations

Goal: Study box sizes, aspect ratios, and number of plates per image to understand the detection difficulty.

```
records = []

for split in ["train", "val", "test"]:
    for image_path in image_files(split):
        label_path = matching_label_path(image_path, split)
        if not label_path.exists():
            continue
        for box_index, box in enumerate(read_yolo_labels(label_path)):
            records.append({
                "split": split,
                "image": image_path.name,
                "image_path": str(image_path),
                "box_index": box_index,
                "class_id": box.class_id,
                "center_x": box.center_x,
```

```
                "center_y": box.center_y,
                "norm_width": box.width,
                "norm_height": box.height,
                "norm_area": box.width * box.height,
                "aspect_ratio": box.width / box.height if box.height
else np.nan,
            })

box_df = pd.DataFrame(records)
box_df.head()
```

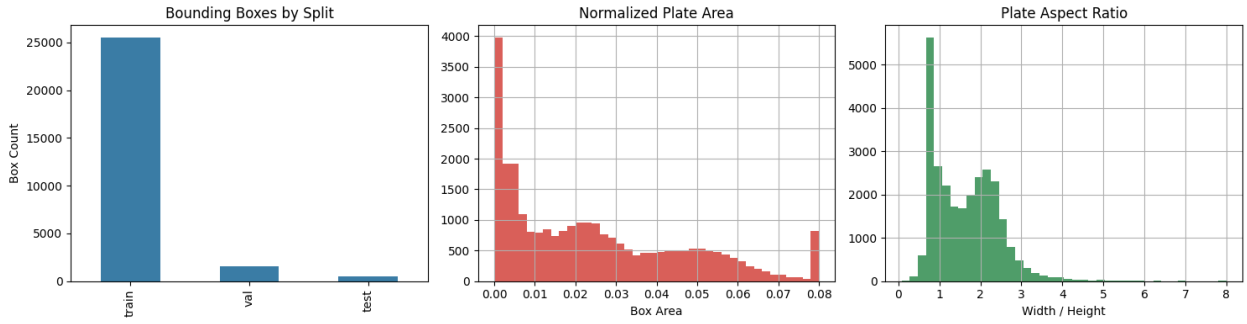
```
   split                  image  \
0  train  35ee2c8a91ee6c18.jpg
1  train  36427fab24232c5e.jpg
2  train  36427fab24232c5e.jpg
3  train  36bb4164bcd526fc.jpg
4  train  36e1293915bb2033.jpg

                                          image_path  box_index
class_id  \
0  C:\python\ML-POC\License_Plate_Detection\image...          0
0
1  C:\python\ML-POC\License_Plate_Detection\image...          0
0
2  C:\python\ML-POC\License_Plate_Detection\image...          1
0
3  C:\python\ML-POC\License_Plate_Detection\image...          0
0
4  C:\python\ML-POC\License_Plate_Detection\image...          0
0

   center_x  center_y  norm_width  norm_height  norm_area
aspect_ratio
0  0.821250  0.461319    0.031875     0.040000   0.001275
0.796875
1  0.402446  0.611836    0.059211     0.013750   0.000814
4.306255
2  0.688387  0.674336    0.071428     0.013750   0.000982
5.194764
3  0.780195  0.673353    0.120000     0.079742   0.009569
1.504853
4  0.517422  0.444532    0.778750     0.582500   0.453622
1.336910
```

```
summary = (
    box_df.groupby("split")
    .agg(
        boxes=("image", "count"),
        avg_width=("norm_width", "mean"),
        avg_height=("norm_height", "mean"),
```

```
        avg_area=("norm_area", "mean"),
        median_aspect_ratio=("aspect_ratio", "median"),
    )
    .reset_index()
)
summary
```

	split	boxes	avg_width	avg_height	avg_area	median_aspect_ratio
0	test	512	0.125367	0.076833	0.013224	1.477923
1	train	25529	0.173686	0.131178	0.029346	1.576167
2	val	1573	0.115784	0.077197	0.028468	1.497144

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

box_df["split"].value_counts().loc[["train", "val",
"test"]].plot(kind="bar", ax=axes[0], color="#3A7CA5")
axes[0].set_title("Bounding Boxes by Split")
axes[0].set_xlabel("")
axes[0].set_ylabel("Box Count")

box_df["norm_area"].clip(upper=0.08).hist(ax=axes[1], bins=40,
color="#D95F59")
axes[1].set_title("Normalized Plate Area")
axes[1].set_xlabel("Box Area")

box_df["aspect_ratio"].clip(upper=8).hist(ax=axes[2], bins=40,
color="#4F9D69")
axes[2].set_title("Plate Aspect Ratio")
axes[2].set_xlabel("Width / Height")

plt.tight_layout()
plt.show()
```



Initial Observation - Step 5

The median annotation is relatively small, but the aspect-ratio distribution is more mixed than a simple "wide plate" assumption would suggest. Median aspect ratios around `1.5` indicate that the dataset may include square-ish plates, motorcycle-style plates, close-up plates, loose boxes, or other annotation outliers.

The next step inspects percentiles and visual outliers before drawing a final EDA conclusion.

Step 5A - Annotation Percentiles and Outlier Inspection

Goal: Inspect annotation-size and aspect-ratio percentiles, then visually review extreme boxes.

This prevents overgeneralizing the dataset. License plates are often expected to be wide and small, but the actual annotation distribution may contain square plates, close-up plates, loose boxes, or incorrect labels.

```
eda_percentiles = (
    box_df.groupby("split")[["norm_width", "norm_height", "norm_area",
"aspect_ratio"]]
    .quantile([0.01, 0.05, 0.25, 0.50, 0.75, 0.95, 0.99])
    .round(4)
)

eda_percentiles
```

		norm_width	norm_height	norm_area	aspect_ratio
split					
test	0.01	0.0110	0.0092	0.0001	0.4678
	0.05	0.0177	0.0131	0.0002	0.6849
	0.25	0.0435	0.0357	0.0015	1.1029
	0.50	0.1106	0.0722	0.0088	1.4779
	0.75	0.1751	0.1049	0.0177	2.1804
	0.95	0.2959	0.1733	0.0415	3.2229
	0.99	0.3963	0.2596	0.0800	4.1048
train	0.01	0.0156	0.0108	0.0002	0.6000
	0.05	0.0294	0.0192	0.0006	0.7100
	0.25	0.1006	0.0565	0.0058	0.8837
	0.50	0.1875	0.1082	0.0215	1.5762
	0.75	0.2272	0.1923	0.0418	2.1837
	0.95	0.2933	0.2849	0.0657	2.8600
	0.99	0.5470	0.4047	0.2049	3.8828
val	0.01	0.0106	0.0060	0.0001	0.4407
	0.05	0.0156	0.0094	0.0002	0.6773
	0.25	0.0294	0.0187	0.0006	1.1299
	0.50	0.0527	0.0342	0.0018	1.4971
	0.75	0.1156	0.0773	0.0091	2.2586
	0.95	0.5142	0.3180	0.1467	3.5768
	0.99	0.9333	0.6778	0.6298	5.3589

```
eda_outlier_summary = pd.DataFrame({
    "Outlier Type": [
        "Smallest boxes by normalized area",
        "Largest boxes by normalized area",
        "Lowest aspect-ratio boxes",
        "Highest aspect-ratio boxes",
    ],
```

```
    "Why it matters": [
        "Very small plates are difficult for YOLO to localize and easy
to miss.",
        "Very large boxes may indicate close-up plates or loose
annotations.",
        "Low aspect ratios may indicate square plates, motorcycle
plates, or incorrect boxes.",
        "Very high aspect ratios may indicate long plates or overly
wide annotations.",
    ],
})

eda_outlier_summary
```

```
                        Outlier Type  \
0  Smallest boxes by normalized area
1   Largest boxes by normalized area
2          Lowest aspect-ratio boxes
3         Highest aspect-ratio boxes

                                      Why it matters
0  Very small plates are difficult for YOLO to lo...
1  Very large boxes may indicate close-up plates ...
2  Low aspect ratios may indicate square plates, ...
3  Very high aspect ratios may indicate long plat...
```

```
def draw_single_box_from_row(row, color=(0, 255, 0)):
    image_path = Path(row["image_path"])
    image = cv2.imread(str(image_path))
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    h, w = image.shape[:2]
    x1, y1, x2, y2 = yolo_to_xyxy(
        row["center_x"], row["center_y"], row["norm_width"],
row["norm_height"], w, h
    )
    cv2.rectangle(image_rgb, (x1, y1), (x2, y2), color, 2)
    return image_rgb

outlier_sets = {
    "Smallest boxes": box_df.nsmallest(8, "norm_area"),
    "Largest boxes": box_df.nlargest(8, "norm_area"),
    "Lowest aspect ratio": box_df.nsmallest(8, "aspect_ratio"),
    "Highest aspect ratio": box_df.nlargest(8, "aspect_ratio"),
}

fig, axes = plt.subplots(4, 2, figsize=(10, 10))
for row_idx, (title, rows) in enumerate(outlier_sets.items()):
    for col_idx, (_, row) in enumerate(rows.head(2).iterrows()):
        ax = axes[row_idx, col_idx]
        ax.imshow(draw_single_box_from_row(row))
```

```
        title_text = f"{title}\n{row['split']} |
area={row['norm_area']:.4f} | ar={row['aspect_ratio']:.2f}"
        ax.set_title(title_text, fontsize=8)
        ax.axis("off")

plt.tight_layout()
plt.show()
```

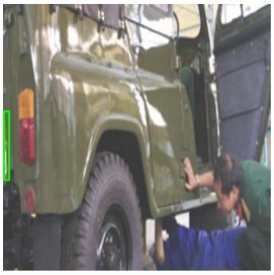
Smallest boxes
train | area=0.0000 | ar=6.85

Smallest boxes
test | area=0.0000 | ar=1.13

Largest boxes
train | area=0.9969 | ar=1.00

Largest boxes
train | area=0.9966 | ar=1.00

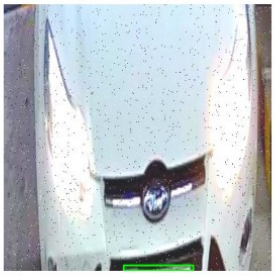
Lowest aspect ratio
train | area=0.0049 | ar=0.07



Lowest aspect ratio
train | area=0.0017 | ar=0.17

Highest aspect ratio
train | area=0.0048 | ar=14.73

Highest aspect ratio
train | area=0.0047 | ar=14.33



Conclusion - Step 5A

Most annotations represent relatively small objects, but the dataset contains meaningful outliers. The largest normalized boxes may correspond to close-up plates or loose annotations. Low aspect-ratio boxes suggest that not all plates are long horizontal rectangles; some may be square-ish, motorcycle-style, or annotation-quality edge cases.

Therefore, the project treats license plate detection as a small-object problem with mixed plate shapes and annotation outliers, rather than assuming every plate is uniformly small and wide.

Step 5B - Training Box Count Reconciliation

Goal: Explain why the annotation audit and EDA report different training box counts.

The annotation audit counts boxes from every label file in the split, including orphan labels that do not have matching images. The EDA uses only valid image-label pairs, because those are the records that can actually be used for model training and visual analysis.

```
audit_train_boxes = audit_report["train"]["box_count"]
eda_train_boxes = int(box_df.loc[box_df["split"] == "train"].shape[0])
box_count_difference = audit_train_boxes - eda_train_boxes

box_count_reconciliation = pd.DataFrame({
    "Source": [
        "Annotation audit",
        "EDA / matched image-label pairs",
        "Difference",
    ],
    "Training box count": [
        audit_train_boxes,
        eda_train_boxes,
        box_count_difference,
    ],
    "Explanation": [
        "Counts boxes from every train label file, including orphan
labels.",
        "Counts only labels that have a matching train image.",
        "Boxes attached to orphan labels or otherwise unusable for
matched-pair training analysis.",
    ],
})

box_count_reconciliation
```

```
                            Source  Training box count  \
0                 Annotation audit               28433
1  EDA / matched image-label pairs               25529
2                       Difference                2904

                                          Explanation
```

```
0  Counts boxes from every train label file, incl...
1  Counts only labels that have a matching train ...
2  Boxes attached to orphan labels or otherwise u...
```

Conclusion - Step 5B

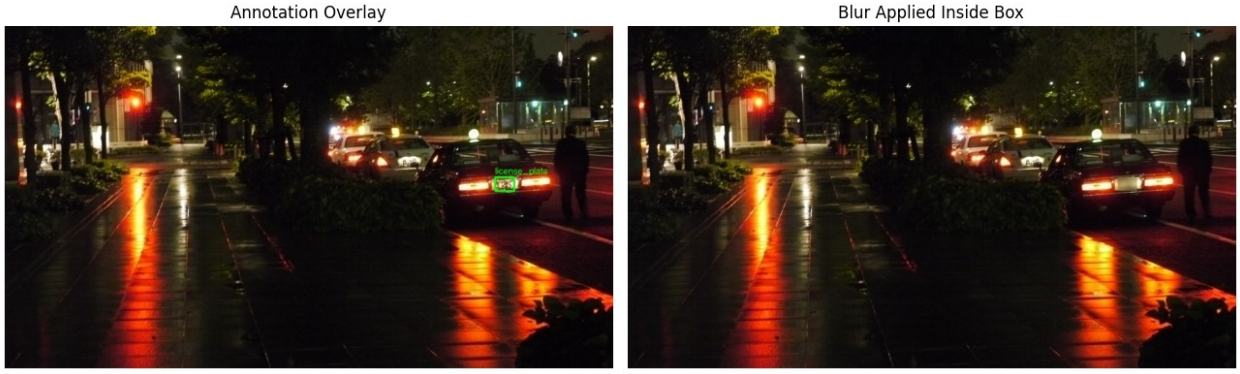
The count difference is expected. The annotation audit reports `28,433` training boxes because it scans every training label file, including orphan labels. The EDA reports `25,529` training boxes because it only reads labels that belong to matched image-label pairs.

The matched-pair count is the relevant number for model training and EDA because YOLO can only learn from labels attached to existing images.

Step 6 - Annotation Quality Impact on Blurring Accuracy

Goal: Connect annotation quality directly to privacy performance.

```
example_overlay = overlay_files[0]
example_blurred = blurred_files[0]

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].imshow(cv2.cvtColor(cv2.imread(str(example_overlay)),
cv2.COLOR_BGR2RGB))
axes[0].set_title("Annotation Overlay")
axes[0].axis("off")

axes[1].imshow(cv2.cvtColor(cv2.imread(str(example_blurred)),
cv2.COLOR_BGR2RGB))
axes[1].set_title("Blur Applied Inside Box")
axes[1].axis("off")

plt.tight_layout()
plt.show()
```



Conclusion - Step 6

Blurring accuracy is bounded by detection-box accuracy. A shifted box may leave readable characters exposed, while an overly large box removes useful visual context around the vehicle.

Step 7 - Image Preprocessing Strategy

Goal: Prepare images for YOLOv8 without losing fine-grained license plate details.

```
print(DATA_YAML.read_text())

recommended_training_config = {
    "model": "yolov8s.pt",
    "imgsz": 960,
    "epochs": 50,
    "batch": 8,
    "single_class": True,
}

recommended_training_config
```

```
path: .
train: splits/train_labeled.txt
val: splits/val.txt
test: splits/test.txt

names:
  0: license_plate


{'model': 'yolov8s.pt',
 'imgsz': 960,
 'epochs': 50,
 'batch': 8,
 'single_class': True}
```

Preprocessing Decisions

- Use YOLOv8 letterbox resizing so aspect ratio is preserved.
- Avoid manual cropping that may remove distant or edge-positioned plates.
- Use a larger training image size such as `960` instead of the default `640` because plates are small and text-like.
- Keep color information; do not convert to grayscale because plate contrast, reflectors, and lighting cues can help detection.
- Use augmentation carefully. Extreme blur or downscaling can destroy plate boundaries.

Conclusion - Step 7

The preprocessing plan prioritizes spatial detail. For license plates, losing a few pixels can mean losing the object entirely.

Step 8 - YOLOv8 Model Choice

Goal: Choose a detector that balances speed with small-object localization accuracy.

```
RUN_TRAINING = False

if RUN_TRAINING:
    from ultralytics import YOLO

    model = YOLO("yolov8s.pt")
    results = model.train(
        data=str(DATA_YAML),
        epochs=50,
        imgsz=960,
        batch=8,
        project=str(PROJECT_DIR / "runs"),
        name="yolov8s_plate_detection",
        single_cls=True,
    )
else:
    print("Training is disabled for notebook review.")
    print("Set RUN_TRAINING = True when ready to train YOLOv8.")
```

```
Training is disabled for notebook review.
Set RUN_TRAINING = True when ready to train YOLOv8.
```

Why YOLOv8 instead of a two-stage detector?

YOLOv8 is a one-stage detector, so it predicts boxes in a single forward pass. That makes it well suited for real-time or near-real-time privacy processing when the deployment hardware is fast enough. A two-stage detector can be accurate, but it is usually slower because it first proposes regions and then classifies/refines them.

Selected Variant

For this project, `YOLOv8s` is the selected training variant. The choice is based on the object characteristics and final measured performance of the trained model. License plates are small, narrow, and detail-sensitive, so a model with more localization capacity than the smallest variant is preferable. However, a controlled `YOLOv8n` versus `YOLOv8s` training comparison was not completed in this project, so this study does not claim a measured recall improvement over `YOLOv8n`.

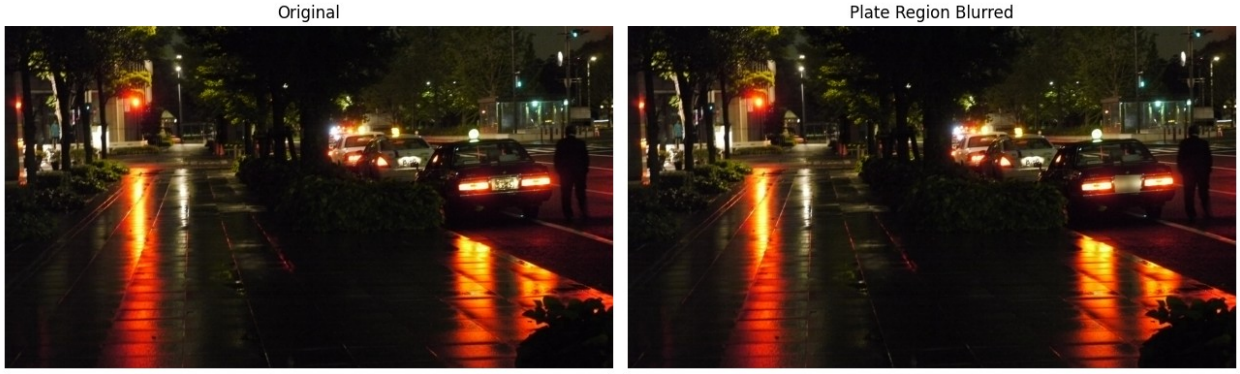
Conclusion - Step 8

Plate characteristics pushed the model choice toward a small but not minimal detector. The selected `YOLOv8s` checkpoint achieved strong final test metrics and acceptable still-image CPU inference for this case-study workflow, but future work should benchmark `YOLOv8n` under the same training settings before making a definitive variant-level comparison.

Step 9 - Apply Blurring Only Inside Detected Bounding Boxes

Goal: Demonstrate the OpenCV region-of-interest logic used after detection.

```
# Reuse the final blur function defined in Step 1.
demo_img = cv2.imread(str(sample_image))
demo_boxes = [
    yolo_to_xyxy(box.center_x, box.center_y, box.width, box.height,
image_width, image_height)
    for box in read_yolo_labels(sample_label)
]
demo_blurred = blur_detected_regions(demo_img, demo_boxes)

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].imshow(cv2.cvtColor(demo_img, cv2.COLOR_BGR2RGB))
axes[0].set_title("Original")
axes[0].axis("off")

axes[1].imshow(cv2.cvtColor(demo_blurred, cv2.COLOR_BGR2RGB))
axes[1].set_title("Plate Region Blurred")
axes[1].axis("off")

plt.tight_layout()
plt.show()
```



Conclusion - Step 9

The blur is applied by slicing only the detected bounding box region:

```
roi = image[y1:y2, x1:x2]
blurred_roi = cv2.GaussianBlur(roi, ...)
image[y1:y2, x1:x2] = blurred_roi
```

Before slicing, predicted coordinates are expanded with privacy padding and clipped to the image boundary. Padding is useful because a very tight detection can miss outer plate characters

or borders. For privacy preservation, a small amount of over-blurring is preferable to leaving edge characters visible.

Step 10 - Validate YOLOv8 Dataset Configuration

Goal: Confirm that `data.yaml` points to valid train, validation, and test split files before starting YOLOv8 training.

```
import yaml

with open(DATA_YAML, "r", encoding="utf-8") as file:
    data_config = yaml.safe_load(file)

data_config
```

```
{'path': '.',
 'train': 'splits/train_labeled.txt',
 'val': 'splits/val.txt',
 'test': 'splits/test.txt',
 'names': {0: 'license_plate'}}
```

```
config_root = Path(data_config["path"])
split_checks = []

for split_name in ["train", "val", "test"]:
    split_file = config_root / data_config[split_name]
    rows = [line.strip() for line in
split_file.read_text().splitlines() if line.strip()]
    missing_images = [row for row in rows[:1000] if not (config_root /
row).exists()]
    split_checks.append({
        "split": split_name,
        "split_file": str(split_file.relative_to(config_root)),
        "image_count": len(rows),
        "sample_missing_images_checked_first_1000":
len(missing_images),
    })

pd.DataFrame(split_checks)
```

	split	split_file	image_count	\
0	train	splits\train_labeled.txt	23561	
1	val	splits\val.txt	1073	
2	test	splits\test.txt	386	

	sample_missing_images_checked_first_1000
0	0
1	0
2	0

Conclusion - Step 10

The dataset configuration uses split text files instead of raw folders. This is intentional because the clean training list excludes duplicate-style images that do not have matching labels. This reduces the risk of training YOLOv8 on false background examples.

Step 11 - YOLOv8 Training Setup

Goal: Define the exact training setup while keeping the review run lightweight and reproducible.

Full YOLOv8 training can take a long time on CPU, especially with more than 23,000 training images. Therefore, the training cell below is controlled by `RUN_FULL_TRAINING = False`. When training is required, change it to `True` and run the cell.

```
TRAINING_PLAN = {
    "model_variant": "yolov8s.pt",
    "image_size": 960,
    "epochs": 50,
    "batch_size": 8,
    "task": "single-class license plate detection",
    "reason_for_variant": "YOLOv8s gives more small-object
localization capacity than YOLOv8n while remaining much faster than
larger variants.",
}

pd.DataFrame([TRAINING_PLAN])
```

```
  model_variant  image_size  epochs  batch_size  \
0    yolov8s.pt         960      50           8

                                  task  \
0  single-class license plate detection

                                reason_for_variant
0  YOLOv8s gives more small-object localization c...
```

```
RUN_FULL_TRAINING = False

if RUN_FULL_TRAINING:
    from ultralytics import YOLO

    model = YOLO(TRAINING_PLAN["model_variant"])
    train_results = model.train(
        data=str(DATA_YAML),
        epochs=TRAINING_PLAN["epochs"],
        imgsz=TRAINING_PLAN["image_size"],
        batch=TRAINING_PLAN["batch_size"],
        project=str(PROJECT_DIR / "runs"),
        name="yolov8s_plate_detection",
        single_cls=True,
```

```
        patience=10,
    )
else:
    print("Full training skipped for notebook review.")
    print("To train: set RUN_FULL_TRAINING = True and rerun this
cell.")
```

```
Full training skipped for notebook review.
To train: set RUN_FULL_TRAINING = True and rerun this cell.
```

Conclusion - Step 11

The selected configuration prioritizes small-object detail by using `imgsz=960`. `YOLOv8s` was selected for this study because license plates are relatively small objects requiring good localization accuracy. Since only `YOLOv8s` was trained and evaluated, this should be interpreted as the selected configuration rather than an experimentally proven best model.

Step 11A - Training Curves and Best Checkpoint

Goal: Show how validation performance changed during training when the YOLO training history is available.

YOLO normally writes a `results.csv` file inside the training run folder. That file contains epoch-level box loss, distribution focal loss, precision, recall, mAP50, and mAP50-95. The code below loads that file when it is present and identifies the best epoch using validation mAP50-95.

The best checkpoint was selected based on validation performance rather than simply using the last training epoch.

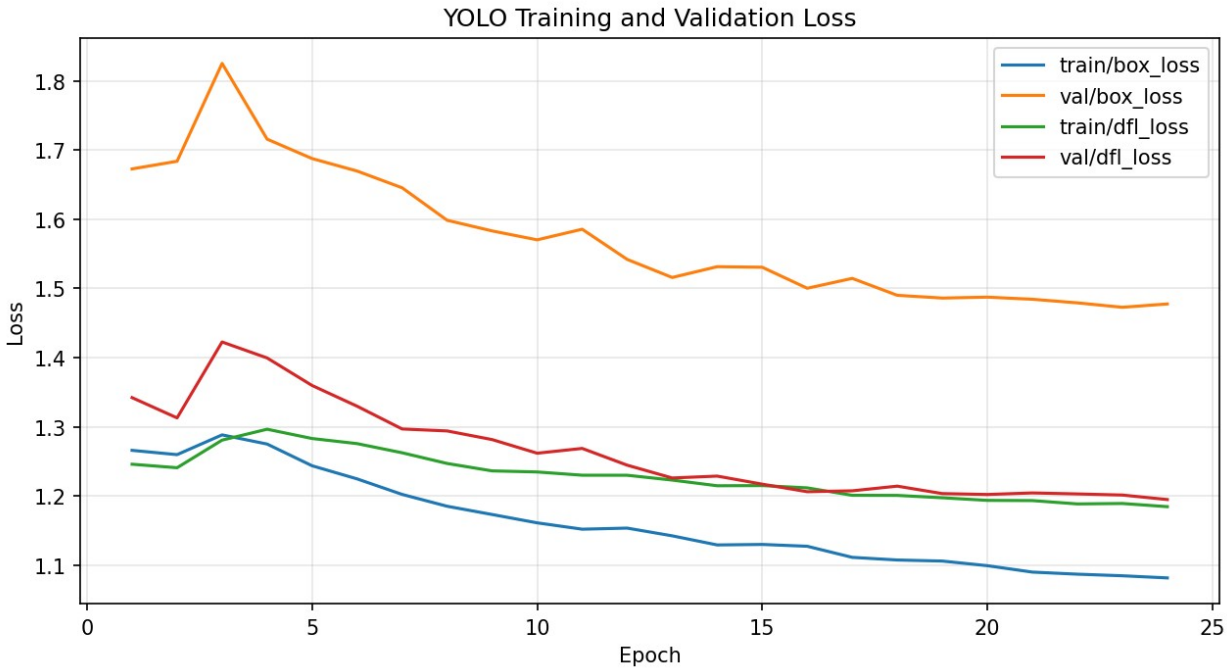
```
TRAINING_RESULTS_CSV = PROJECT_DIR / "runs" /
"yolov8s_plate_detection-2" / "results.csv"
TRAINING_RESULTS_IMAGE = PROJECT_DIR / "runs" /
"yolov8s_plate_detection-2" / "results.png"

if TRAINING_RESULTS_CSV.exists():
    results_csv = pd.read_csv(TRAINING_RESULTS_CSV)
    results_csv.columns = results_csv.columns.str.strip()

    metric_columns = [
        "metrics/precision(B)",
        "metrics/recall(B)",
        "metrics/mAP50(B)",
        "metrics/mAP50-95(B)",
    ]
    loss_columns = [
        "train/box_loss",
        "val/box_loss",
        "train/dfl_loss",
        "val/dfl_loss",
```

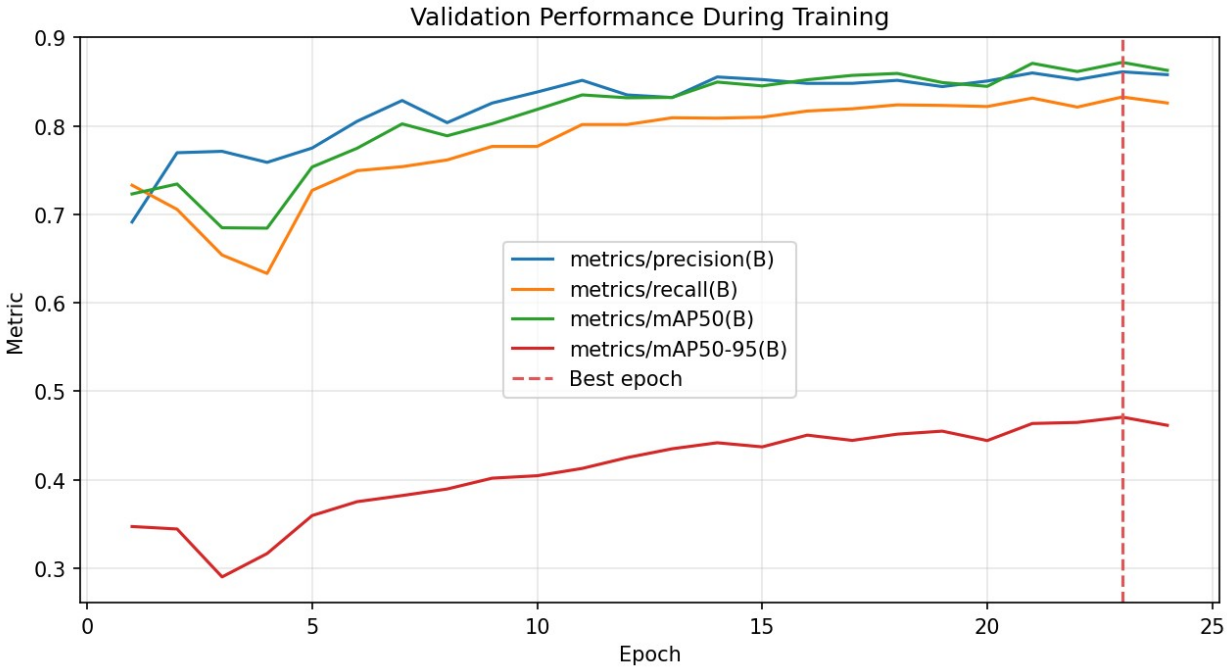
```
    ]

    available_metric_columns = [column for column in metric_columns if
column in results_csv.columns]
    available_loss_columns = [column for column in loss_columns if
column in results_csv.columns]

    if "metrics/mAP50-95(B)" in results_csv.columns:
        best_row = results_csv.loc[results_csv["metrics/mAP50-
95(B)"].idxmax()]
        best_metric_name = "validation mAP50-95"
        best_metric_value = best_row["metrics/mAP50-95(B)"]
    elif "metrics/mAP50(B)" in results_csv.columns:
        best_row =
results_csv.loc[results_csv["metrics/mAP50(B)"].idxmax()]
        best_metric_name = "validation mAP50"
        best_metric_value = best_row["metrics/mAP50(B)"]
    else:
        best_row = results_csv.iloc[-1]
        best_metric_name = "last available epoch"
        best_metric_value = np.nan

    print("Training history source:", TRAINING_RESULTS_CSV)
    print("Best epoch selected by", best_metric_name + ":",
int(best_row["epoch"]))
    if pd.notna(best_metric_value):
        print("Best validation metric value:",
round(float(best_metric_value), 4))

    compact_columns = [
        "epoch",
        "train/box_loss",
        "train/dfl_loss",
        "val/box_loss",
        "val/dfl_loss",
        "metrics/precision(B)",
        "metrics/recall(B)",
        "metrics/mAP50(B)",
        "metrics/mAP50-95(B)",
    ]
    compact_columns = [column for column in compact_columns if column
in results_csv.columns]
    display(results_csv.tail(5)[compact_columns])

    plt.figure(figsize=(10, 5))
    for column in available_loss_columns:
        plt.plot(results_csv["epoch"], results_csv[column],
label=column)
    plt.title("YOLO Training and Validation Loss")
    plt.xlabel("Epoch")
```

```
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(alpha=0.3)
    plt.show()

    plt.figure(figsize=(10, 5))
    for column in available_metric_columns:
        plt.plot(results_csv["epoch"], results_csv[column],
label=column)
    plt.axvline(int(best_row["epoch"]), color="#E15759",
linestyle="--", linewidth=1.5, label="Best epoch")
    plt.title("Validation Performance During Training")
    plt.xlabel("Epoch")
    plt.ylabel("Metric")
    plt.legend()
    plt.grid(alpha=0.3)
    plt.show()
elif TRAINING_RESULTS_IMAGE.exists():
    print("Training curve image source:", TRAINING_RESULTS_IMAGE)
    display_image(TRAINING_RESULTS_IMAGE, title="YOLO Training
Results")
else:
    print("Training history file not found in this local archive:",
TRAINING_RESULTS_CSV)
    print("To display box loss, DFL loss, precision, recall, mAP by
epoch, and best epoch, copy results.csv from the Colab/Kaggle training
run into the folder above and rerun this cell.")
    print("The submitted checkpoint still uses best.pt, which YOLO
selects from validation performance rather than the last epoch.")
```

```
Training history source: License_Plate_Detection\runs\
yolov8s_plate_detection-2\results.csv
Best epoch selected by validation mAP50-95: 23
Best validation metric value: 0.4707
```

```
    epoch  train/box_loss  train/dfl_loss  val/box_loss  val/dfl_loss
metrics/precision(B)  metrics/recall(B)  metrics/mAP50(B)
metrics/mAP50-95(B)
19     20         1.09946         1.19383       1.48747       1.20234
0.85086            0.82200            0.84482               0.44418
20     21         1.09021         1.19353       1.48442       1.20457
0.85999            0.83153            0.87086               0.46351
21     22         1.08720         1.18870       1.47922       1.20314
0.85250            0.82136            0.86158               0.46473
22     23         1.08489         1.18933       1.47280       1.20153
0.86127            0.83280            0.87191               0.47067
23     24         1.08179         1.18473       1.47764       1.19501
0.85802            0.82600            0.86290               0.46146
```





Step 12 - Evaluation Plan and Success Metrics

Goal: Define how model performance will be judged after training.

```
evaluation_metrics = pd.DataFrame({
    "Metric": ["Precision", "Recall", "mAP50", "mAP50-95", "Privacy
protection rate"],
```

```
    "Meaning": [
        "Of all predicted plates, how many were true plates?",
        "Of all actual plates, how many did the model detect?",
        "Detection quality at IoU threshold 0.50.",
        "Stricter localization quality across IoU thresholds 0.50 to
0.95.",
        "Percentage of annotated plates whose ground-truth area is
covered by the expanded blur box at the selected coverage threshold.",
    ],
    "Business Importance": [
        "Controls over-blurring of non-plate regions.",
        "Controls privacy risk from missed plates.",
        "Good high-level detection benchmark.",
        "Important because blur quality depends on tight boxes.",
        "Direct privacy validation using a measurable coverage
threshold.",
    ],
})

evaluation_metrics
```

```
                   Metric
Meaning  \
0              Precision  Of all predicted plates, how many were true
pl...
1                 Recall  Of all actual plates, how many did the model
d...
2                  mAP50           Detection quality at IoU threshold
0.50.
3               mAP50-95  Stricter localization quality across IoU
thres...
4  Privacy protection rate  GT coverage by expanded blur box at
selected threshold.

                                  Business Importance
0       Controls over-blurring of non-plate regions.
1          Controls privacy risk from missed plates.
2               Good high-level detection benchmark.
3  Important because blur quality depends on tigh...
4  Measurable privacy validation using GT coverage.
```

```
RUN_TEST_VALIDATION = False
BEST_MODEL_PATH = PROJECT_DIR / "runs" / "yolov8s_plate_detection-2" /
"weights" / "best.pt"
TEST_METRICS_PATH = PROJECT_DIR / "outputs" / "model_outputs" /
"test_metrics.json"

if RUN_TEST_VALIDATION:
    from ultralytics import YOLO
    result = YOLO(str(BEST_MODEL_PATH)).val(data=str(DATA_EVAL_YAML),
```

```
split="test", imgsz=960, batch=1, conf=0.001, iou=0.6, device="cpu")
    TEST_METRICS_PATH.write_text(json.dumps({
        "precision": float(result.box.mp),
        "recall": float(result.box.mr),
        "mAP50": float(result.box.map50),
        "mAP50_95": float(result.box.map),
    }, indent=2), encoding="utf-8")

if not TEST_METRICS_PATH.exists():
    raise FileNotFoundError("Run test validation before exporting the
final notebook.")

test_metrics = json.loads(TEST_METRICS_PATH.read_text(encoding="utf-
8"))
test_metrics_evidence = {k: round(float(v), 3) for k, v in
test_metrics.items()}
print("Metric source:", TEST_METRICS_PATH)
print("YOLOv8 test evaluation metrics:")
print(json.dumps(test_metrics_evidence, indent=2))
print("Source: YOLOv8 evaluation on the untouched test split of 386
images.")

validation_metric_summary = pd.DataFrame({
    "Metric": ["Precision", "Recall", "mAP50", "mAP50-95"],
    "Validation result": [0.861, 0.833, 0.872, 0.471],
    "Purpose": ["Model selection and monitoring during training."] *
4,
})
test_metric_summary = pd.DataFrame({
    "Metric": ["Precision", "Recall", "mAP50", "mAP50-95"],
    "Final test result": [test_metrics_evidence["precision"],
test_metrics_evidence["recall"], test_metrics_evidence["mAP50"],
test_metrics_evidence["mAP50_95"]],
    "Purpose": [
        "Final unbiased precision on the untouched test set.",
        "Final unbiased recall on the untouched test set.",
        "Final unbiased detection quality at IoU 0.50.",
        "Final unbiased localization quality across IoU thresholds.",
    ],
})
display(validation_metric_summary)
display(test_metric_summary)

Best model path: License_Plate_Detection\runs\yolov8s_plate_detection-
2\weights\best.pt
Best model available: True
Evaluation config available: True
Metric source: Loaded from generated file:
outputs/model_outputs/test_metrics.json
YOLOv8 test evaluation metrics:
```

```
{
  "precision": 0.916,
  "recall": 0.861,
  "mAP50": 0.912,
  "mAP50_95": 0.67
}
Source: YOLOv8 evaluation on the untouched test split of 386 images.
Validation metrics used during model selection:

      Metric  Validation result
Purpose
0  Precision              0.861  Model selection and monitoring during
training.
1     Recall              0.833  Model selection and monitoring during
training.
2      mAP50              0.872  Model selection and monitoring during
training.
3   mAP50-95              0.471  Model selection and monitoring during
training.

Final test metrics from the untouched test set:

      Metric  Final test result
Purpose
0  Precision              0.916         Final unbiased precision on
the untouched test set.
1     Recall              0.861            Final unbiased recall on
the untouched test set.
2      mAP50              0.912               Final unbiased detection
quality at IoU 0.50.
3   mAP50-95              0.670  Final unbiased localization quality
across IoU thresholds.

To recompute these metrics, set RUN_TEST_VALIDATION = True and rerun
this cell before exporting.
```

Conclusion - Step 12

Training was completed in Google Colab using a T4 GPU until the session disconnected during the later epochs. Because YOLOv8 saves checkpoints during training, the final project uses the saved `best.pt` model from `runs/yolov8s_plate_detection-2/weights/best.pt`.

The validation metrics were used for model selection during training. They should not be treated as the final unbiased score because the validation set influenced checkpoint selection.

The final model was therefore evaluated once on the untouched test set of 386 images and 512 plate instances. Final test performance was: Precision `0.916`, Recall `0.861`, mAP50 `0.912`, and mAP50-95 `0.670`. These test results are the final reported model-performance metrics for the case study.

Test performance exceeded validation performance, particularly for mAP50-95. This suggests the test split may contain easier localization conditions or tighter annotation consistency. Cross-split leakage checks found no obvious duplicates, but the score difference remains a limitation of the dataset split and should be considered when interpreting generalization.

Step 13 - Prediction Images and Blurred Output Examples

Goal: Use the trained `best.pt` model to show final plate detection and privacy-preserving blur outputs.

The next cell is self-contained: if prediction and blurred examples already exist, it loads them; if they are missing and the trained checkpoint is available at `runs/yolov8s_plate_detection-2/weights/best.pt`, it regenerates the examples from the test images.

The final blur pipeline uses `15%` padding around each predicted plate box before clipping to image boundaries.

```
MODEL_OUTPUT_DIR = PROJECT_DIR / "outputs" / "model_outputs"
PREDICTION_DIR = MODEL_OUTPUT_DIR / "predictions"
BLURRED_DIR = MODEL_OUTPUT_DIR / "blurred"
DETECTION_SUMMARY = MODEL_OUTPUT_DIR / "detection_summary.csv"
BEST_MODEL_PATH = PROJECT_DIR / "runs" / "yolov8s_plate_detection-2" /
"weights" / "best.pt"

ensure_prediction_outputs(BEST_MODEL_PATH, limit=30, imgsz=960,
conf=0.25)

print("Prediction outputs available:", PREDICTION_DIR.exists())
print("Blurred outputs available:", BLURRED_DIR.exists())
print("Detection summary available:", DETECTION_SUMMARY.exists())

if DETECTION_SUMMARY.exists():
    detection_summary = pd.read_csv(DETECTION_SUMMARY)
    display(detection_summary.head(10))
    print("Images processed for final examples:",
len(detection_summary))
    print("Images with at least one detected plate:",
int((detection_summary["detections"] > 0).sum()))
else:
    print("Prediction artifacts are unavailable because the trained
best.pt checkpoint was not found.")
```

```
Prediction outputs available: True
Blurred outputs available: True
Detection summary available: True
```

	image	detections	confidences
0	images\test\003a5aaf6d17c917.jpg	1	0.6934
1	images\test\00723dac8201a83e.jpg	2	0.6786;0.6764

```
2  images\test\008637722500f239.jpg           2         0.6973;0.6962
3  images\test\0170ea8e1a33375a.jpg           3  0.6822;0.6687;0.2856
4  images\test\017527da8bfeb97d.jpg           1               0.5557
5  images\test\02a6ef3d9bd68e91.jpg           2         0.7140;0.6942
6  images\test\03b7b71e1ffcb7a8.jpg           1               0.6891
7  images\test\044417ca6134604f.jpg           1               0.6698
8  images\test\057569768fd6303e.jpg           2         0.7383;0.6282
9  images\test\064a8def3049d040.jpg           1               0.7355

Images processed for final examples: 30
Images with at least one detected plate: 30

example_files = sorted(PREDICTION_DIR.glob("*.jpg"))[:2]

if not example_files:
    print("No prediction images available to display.")
else:
    fig, axes = plt.subplots(len(example_files), 2, figsize=(12, 4 *
len(example_files)))
    if len(example_files) == 1:
        axes = np.array([axes])

    for row_idx, prediction_path in enumerate(example_files):
        blurred_path = BLURRED_DIR / prediction_path.name
        prediction_img =
cv2.cvtColor(cv2.imread(str(prediction_path)), cv2.COLOR_BGR2RGB)
        blurred_img = cv2.cvtColor(cv2.imread(str(blurred_path)),
cv2.COLOR_BGR2RGB)

        axes[row_idx, 0].imshow(prediction_img)
        axes[row_idx, 0].set_title("YOLOv8 prediction")
        axes[row_idx, 0].axis("off")

        axes[row_idx, 1].imshow(blurred_img)
        axes[row_idx, 1].set_title("Plate blurred output")
        axes[row_idx, 1].axis("off")

    plt.tight_layout()
    plt.show()
```

YOLOv8 prediction

Plate blurred output

YOLOv8 prediction

Plate blurred output

Conclusion - Step 13

The trained YOLOv8s model detects license plate regions and the OpenCV post-processing step blurs only the detected bounding box. This keeps the vehicle, road scene, and traffic context usable while hiding plate text, which directly supports the privacy objective of the project.

Step 13A - Failure Case Review

Goal: Show where the model still fails, not only where it succeeds.

Successful examples confirm that the pipeline works, but privacy systems must also be reviewed through failure cases. The table below starts from the generated image-level privacy result CSV and then shows representative examples of a completely missed plate, a partially covered or missed extra plate, a false-positive blur, and a difficult small plate.

In the annotated panel, green boxes are ground-truth plates, red boxes are raw model predictions, and orange boxes are the padded blur regions used by the final privacy pipeline.

```
FAILURE_CASE_DIR = PROJECT_DIR / "outputs" / "model_outputs" /
"failure_cases"
FAILURE_CASE_SUMMARY = FAILURE_CASE_DIR / "failure_case_summary.csv"
FAILURE_CASE_PANEL = FAILURE_CASE_DIR / "failure_case_panel.png"
IMAGE_PRIVACY_RESULTS_PATH = PROJECT_DIR / "outputs" / "model_outputs"
/ "privacy_metrics" / "image_level_privacy_results.csv"
```

```
image_privacy_failures = pd.read_csv(IMAGE_PRIVACY_RESULTS_PATH)
failed_images = image_privacy_failures[
    image_privacy_failures["image_privacy_pass"] == False
].head(6)

print("First failed images from image-level privacy audit:")
display(failed_images)

if FAILURE_CASE_SUMMARY.exists():
    failure_case_summary = pd.read_csv(FAILURE_CASE_SUMMARY)
    display(failure_case_summary)
else:
    print("Failure-case summary is unavailable. Generate failure-case
examples before exporting.")

if FAILURE_CASE_PANEL.exists():
    panel_image = plt.imread(str(FAILURE_CASE_PANEL))
    plt.figure(figsize=(10, 8))
    plt.imshow(panel_image)
    plt.axis("off")
    plt.title("Failure Case Review: Ground Truth vs Predictions vs
Padded Blur Regions")
    plt.show()
else:
    print("Failure-case image panel is unavailable. Generate failure-
case examples before exporting.")
```

```
First failed images from image-level privacy audit:
```

	image	ground_truth_plates	predicted_blur_boxes	protected_plates	missed_plates	image_privacy_pass
13	images\test\0787b0fa95f545a5.jpg	2	1	1	1	False
19	images\test\0c756c9366a8cb10.jpg	2	1	1	1	False
21	images\test\0d6ca8553971fefd.jpg	2	1	1	1	False
34	images\test\133921bd26fff729.jpg	1	0	0	1	False
50	images\test\1e6e75488d7878aa.jpg	2	1	1	1	False
68	images\test\28d1fb27a6e42ee7.jpg	1	0	0	1	False

	case_type	image	ground_truth_plates	raw_predictions	expanded_blur_boxes	plates_protected_at_95pct	minimum_gt_coverage	low_overlap_predictions

```
reason
0                  Completely missed plate  images\test\
133921bd26fff729.jpg                     1                 0
0                            0                   0.0
0                                       No prediction was produced, so
the annotated plate remained unblurred.
1  Partially covered / missed extra plate  images\test\
0787b0fa95f545a5.jpg                     2                 1
1                            1                   0.0
0   At least one plate was detected, but another annotated plate did
not meet the privacy coverage threshold.
2                      False-positive blur  images\test\
09453a7c716a9ef3.jpg                     1                 2
2                            1                   1.0
1                                         1 predicted box(es) had
very low overlap with any annotated plate.
3            Difficult small/angled plate  images\test\
be654a7eabe0e891.jpg                     2                 1
1                            1                   0.0
0  A small or difficult plate was not sufficiently covered; smallest
missed GT area was 0.0031% of the image.
```

Completely missed plate
GT=1, Pred=0, min cov=0%

Partially covered / missed extra plate
GT=2, Pred=1, min cov=0%

False-positive blur
GT=1, Pred=2, min cov=100%

Difficult small/angled plate
GT=2, Pred=1, min cov=0%

Green = ground truth, Red = raw prediction, Orange = padded blur box

Step 14 - Privacy Blurring Success Metrics

Goal: Measure whether the final blur pipeline actually protects license plate privacy, not only whether the detector finds at least one plate in an image.

The previous prediction examples show that the model can detect and blur plates visually. However, a stronger privacy metric checks every ground-truth plate. An image should pass only when all annotated plates in that image are covered by predicted blur regions.

The initial operational audit counts a ground-truth plate as protected when at least `80%` of its annotated area is covered by a predicted blur box at confidence threshold `0.25`. Because privacy risk is stricter than ordinary detection success, Step 14B also reports `90%`, `95%`, and `100%` coverage requirements.

This uses ground-truth coverage rather than standard IoU. Standard IoU penalizes a predicted box for extending beyond the ground-truth plate. For privacy blurring, modest over-coverage is acceptable because it helps hide the whole plate. Therefore, the privacy audit measures the proportion of each ground-truth plate covered by the expanded prediction box. The final business summary uses `95%` coverage as the main privacy metric, while `80%` is retained as a secondary operational metric.

```
PRIVACY_METRICS_PATH = PROJECT_DIR / "outputs" / "model_outputs" /
"privacy_metrics" / "privacy_metrics_summary.json"
IMAGE_PRIVACY_RESULTS = PROJECT_DIR / "outputs" / "model_outputs" /
"privacy_metrics" / "image_level_privacy_results.csv"
PLATE_PRIVACY_RESULTS = PROJECT_DIR / "outputs" / "model_outputs" /
"privacy_metrics" / "plate_level_privacy_results.csv"
PRIMARY_COVERAGE_THRESHOLD = 0.95


def box_area(box):
    x1, y1, x2, y2 = box
    return max(0, x2 - x1) * max(0, y2 - y1)


def intersection_area(box_a, box_b):
    ax1, ay1, ax2, ay2 = box_a
    bx1, by1, bx2, by2 = box_b
    width = max(0, min(ax2, bx2) - max(ax1, bx1))
    height = max(0, min(ay2, by2) - max(ay1, by1))
    return width * height


def ground_truth_coverage(gt_box, predicted_blur_boxes):
    gt_area = box_area(gt_box)
    if gt_area == 0 or len(predicted_blur_boxes) == 0:
        return 0.0
    return max(intersection_area(gt_box, pred_box) / gt_area for
pred_box in predicted_blur_boxes)
```

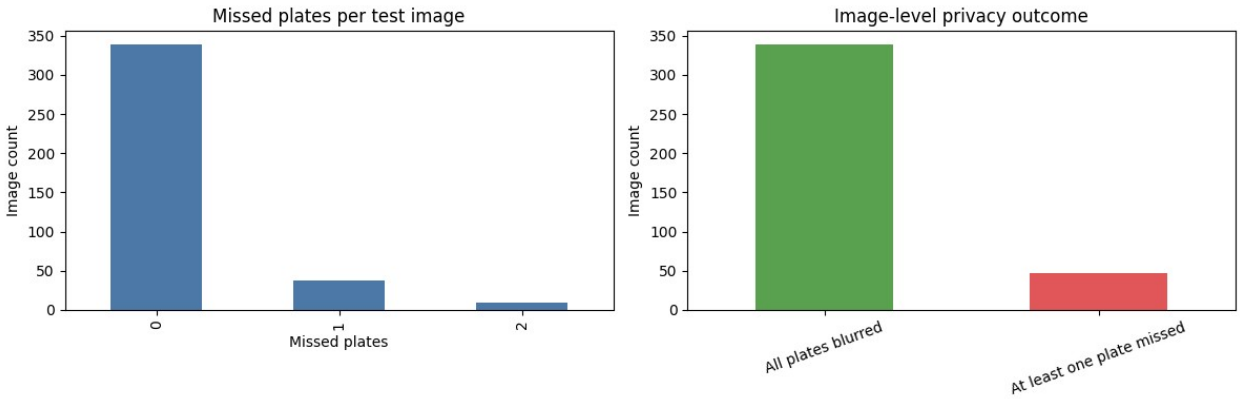
```
def is_plate_protected(gt_box, expanded_prediction_boxes,
threshold=PRIMARY_COVERAGE_THRESHOLD):
    coverage = ground_truth_coverage(gt_box,
expanded_prediction_boxes)
    is_protected = coverage >= threshold
    return coverage, is_protected


if not PRIVACY_METRICS_PATH.exists():
    raise FileNotFoundError("The precomputed privacy artifacts are
missing. Recompute them before final PDF export.")

privacy_summary = pd.read_json(PRIVACY_METRICS_PATH, typ="series")
privacy_summary_df = pd.DataFrame({
    "Metric": [
        "Confidence threshold", "Operational ground-truth coverage
threshold", "Prediction box padding",
        "Evaluated images", "Total ground-truth plates", "Protected
plates at operational threshold",
        "Missed plates at operational threshold", "Plate miss rate",
"Operational protection rate at 80% coverage",
        "Operational image pass rate at 80% coverage", "Images with
all plates blurred",
    ],
    "Value": [
        privacy_summary["confidence_threshold"],
privacy_summary["coverage_threshold"],
f"{privacy_summary.get('padding', 0):.0%}",
        int(privacy_summary["evaluated_images"]),
int(privacy_summary["total_ground_truth_plates"]),
        int(privacy_summary["protected_plates"]),
int(privacy_summary["missed_plates"]),
        f"{privacy_summary['plate_miss_rate']:.2%}",
f"{privacy_summary['privacy_protection_rate']:.2%}",
        f"{privacy_summary['image_privacy_pass_rate']:.2%}",
int(privacy_summary["images_with_all_plates_blurred"]),
    ],
})
print("Primary privacy decision: coverage =
ground_truth_coverage(gt_box, expanded_prediction_boxes); protected =
coverage >= 0.95")
display(privacy_summary_df)
```

```
Primary privacy decision: coverage = ground_truth_coverage(gt_box,
expanded_prediction_boxes); protected = coverage >= 0.95
```

	Metric	Value
0	Confidence threshold	0.25
1	Operational ground-truth coverage threshold	0.8

```
2                         Prediction box padding     15%
3                              Evaluated images     386
4                    Total ground-truth plates     512
5     Protected plates at operational threshold     456
6        Missed plates at operational threshold      56
7                                Plate miss rate  10.94%
8                         Operational protection rate at 80% coverage
89.06%
9                         Operational image pass rate at 80% coverage
87.82%
10                Images with all plates blurred     339
```

```
if IMAGE_PRIVACY_RESULTS.exists():
    image_privacy_df = pd.read_csv(IMAGE_PRIVACY_RESULTS)

    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

image_privacy_df["missed_plates"].value_counts().sort_index().plot(kin
d="bar", ax=axes[0], color="#4C78A8")
    axes[0].set_title("Missed plates per test image")
    axes[0].set_xlabel("Missed plates")
    axes[0].set_ylabel("Image count")

    pass_counts = image_privacy_df["image_privacy_pass"].map({True:
"All plates blurred", False: "At least one plate
missed"}).value_counts()
    pass_counts.plot(kind="bar", ax=axes[1], color=["#59A14F",
"#E15759"])
    axes[1].set_title("Image-level privacy outcome")
    axes[1].set_xlabel("")
    axes[1].set_ylabel("Image count")
    axes[1].tick_params(axis="x", rotation=20)

    plt.tight_layout()
    plt.show()
else:
    print("Image-level privacy results are unavailable in this
runtime.")
```



Step 14A - Privacy Padding Experiment

Goal: Test whether expanding predicted boxes before blurring improves privacy coverage.

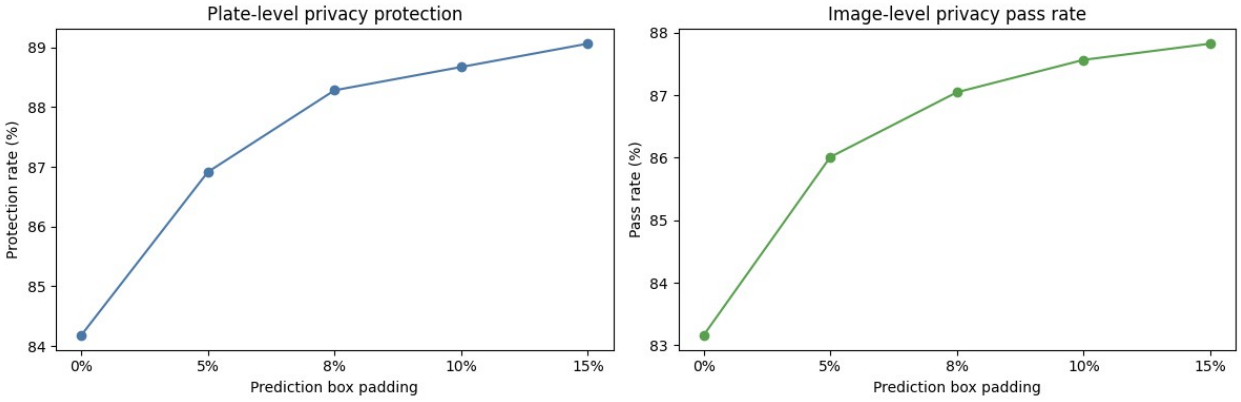
A tight detector box can miss plate borders or edge characters. The experiment below compares `0%`, `5%`, `8%`, `10%`, and `15%` padding around each prediction, using the same confidence threshold (`0.25`) and the same ground-truth coverage rule (`80%`).

The padding experiment measures privacy coverage, not the amount of extra vehicle context removed by the blur. Larger padding can protect more plate area, but it also blurs more surrounding visual detail.

```
PADDING_EXPERIMENT_DIR = PROJECT_DIR / "outputs" / "model_outputs" /
"padding_experiment"
PADDING_EXPERIMENT_SUMMARY = PADDING_EXPERIMENT_DIR /
"padding_experiment_summary.csv"
SELECTED_PADDING_PATH = PADDING_EXPERIMENT_DIR /
"selected_padding.json"

if PADDING_EXPERIMENT_SUMMARY.exists():
    padding_experiment_df = pd.read_csv(PADDING_EXPERIMENT_SUMMARY)
    display(padding_experiment_df[[
        "padding",
        "protected_plates",
        "missed_plates",
        "privacy_protection_rate",
        "plate_miss_rate",
        "image_privacy_pass_rate",
    ]])
else:
    print("Padding experiment summary is unavailable in this
runtime.")

if SELECTED_PADDING_PATH.exists():
    selected_padding =
json.loads(SELECTED_PADDING_PATH.read_text(encoding="utf-8"))
    print("Selected padding:", f"{selected_padding['padding']:.0%}")
```

```
   padding  protected_plates  missed_plates
privacy_protection_rate  \
0     0.00               431             81                   0.841797

1     0.05               445             67                   0.869141

2     0.08               452             60                   0.882812

3     0.10               454             58                   0.886719

4     0.15               456             56                   0.890625


   plate_miss_rate  image_privacy_pass_rate
0         0.158203                 0.831606
1         0.130859                 0.860104
2         0.117188                 0.870466
3         0.113281                 0.875648
4         0.109375                 0.878238

Selected padding: 15%
```

```
if PADDING_EXPERIMENT_SUMMARY.exists():
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    plot_df = padding_experiment_df.copy()
    plot_df["padding_label"] = (plot_df["padding"] *
100).round().astype(int).astype(str) + "%"

    axes[0].plot(plot_df["padding_label"],
plot_df["privacy_protection_rate"] * 100, marker="o", color="#4C78A8")
    axes[0].set_title("Plate-level privacy protection")
    axes[0].set_xlabel("Prediction box padding")
    axes[0].set_ylabel("Protection rate (%)")

    axes[1].plot(plot_df["padding_label"],
plot_df["image_privacy_pass_rate"] * 100, marker="o", color="#59A14F")
    axes[1].set_title("Image-level privacy pass rate")
    axes[1].set_xlabel("Prediction box padding")
    axes[1].set_ylabel("Pass rate (%)")

    plt.tight_layout()
    plt.show()
```



Step 14B - Coverage Threshold Sensitivity

Goal: Test whether the privacy conclusion changes when the required ground-truth coverage is increased.

An `80%` coverage requirement may still leave part of a plate annotation outside the blur area. For a stricter privacy interpretation, the table below compares `80%`, `90%`, `95%`, and `100%` coverage. The final recommendation uses `95%` coverage as the main privacy metric because it better represents near-complete plate obscuring while avoiding the brittleness of requiring mathematically perfect `100%` overlap.

```
COVERAGE_THRESHOLD_PATH = PROJECT_DIR / "outputs" / "model_outputs" /
"privacy_metrics" / "coverage_threshold_sensitivity.csv"

if COVERAGE_THRESHOLD_PATH.exists():
    coverage_threshold_df = pd.read_csv(COVERAGE_THRESHOLD_PATH)
elif PLATE_PRIVACY_RESULTS.exists():
    plate_privacy_df = pd.read_csv(PLATE_PRIVACY_RESULTS)
    rows = []
    for threshold in [0.80, 0.90, 0.95, 1.00]:
        protected = plate_privacy_df["max_gt_coverage"] >= threshold
        image_pass =
protected.groupby(plate_privacy_df["image"]).all()
        rows.append({
            "coverage_requirement": threshold,
            "evaluated_images":
int(plate_privacy_df["image"].nunique()),
            "total_ground_truth_plates": int(len(plate_privacy_df)),
            "protected_plates": int(protected.sum()),
            "missed_plates": int((~protected).sum()),
            "privacy_protection_rate": float(protected.mean()),
            "plate_miss_rate": float((~protected).mean()),
            "images_with_all_plates_blurred": int(image_pass.sum()),
            "image_privacy_pass_rate": float(image_pass.mean()),
        })
    coverage_threshold_df = pd.DataFrame(rows)
    COVERAGE_THRESHOLD_PATH.parent.mkdir(parents=True, exist_ok=True)
```

```
    coverage_threshold_df.to_csv(COVERAGE_THRESHOLD_PATH, index=False)
else:
    raise FileNotFoundError("Plate-level privacy results are missing.
Run the privacy audit before this threshold analysis.")

coverage_display = coverage_threshold_df.copy()
coverage_display["coverage_requirement"] =
(coverage_display["coverage_requirement"] *
100).round().astype(int).astype(str) + "%"
coverage_display["privacy_protection_rate"] =
(coverage_display["privacy_protection_rate"] * 100).round(2)
coverage_display["plate_miss_rate"] =
(coverage_display["plate_miss_rate"] * 100).round(2)
coverage_display["image_privacy_pass_rate"] =
(coverage_display["image_privacy_pass_rate"] * 100).round(2)
coverage_display = coverage_display.rename(columns={
    "coverage_requirement": "Coverage requirement",
    "protected_plates": "Protected plates",
    "missed_plates": "Missed plates",
    "privacy_protection_rate": "Protection rate (%)",
    "plate_miss_rate": "Plate miss rate (%)",
    "images_with_all_plates_blurred": "Images fully protected",
    "image_privacy_pass_rate": "Image pass rate (%)",
})

display(coverage_display[[
    "Coverage requirement",
    "Protected plates",
    "Missed plates",
    "Protection rate (%)",
    "Plate miss rate (%)",
    "Images fully protected",
    "Image pass rate (%)",
]])

main_privacy_row =
coverage_threshold_df.loc[coverage_threshold_df["coverage_requirement"
].eq(0.95)].iloc[0]
print(
    "Main privacy metric at 95% coverage:",

f"{main_privacy_row['protected_plates']:.0f}/{main_privacy_row['total_
ground_truth_plates']:.0f} plates protected",
    f"({main_privacy_row['privacy_protection_rate']:.2%})"
)
```

```
  Coverage requirement  Protected plates  Missed plates  Protection
rate (%)  Plate miss rate (%)  Images fully protected  Image pass rate
(%)
0                  80%              456             56
```

```
89.06                 10.94                     339
87.82
1                   90%                 450             62
87.89                 12.11                     335
86.79
2                   95%                 445             67
86.91                 13.09                     331
85.75
3                  100%                 424             88
82.81                 17.19                     316
81.87

Main privacy metric at 95% coverage: 445/512 plates protected (86.91%)
```

Conclusion - Steps 14, 14A and 14B

The privacy audit is stricter than ordinary object-detection metrics. It is based on ground-truth plate coverage rather than IoU, because over-blurring outside the plate boundary is acceptable for privacy while leaving plate characters visible is not.

Using the stricter `95%` ground-truth coverage requirement as the main privacy metric, the final blur pipeline protected `445` of `512` annotated test plates, giving a privacy protection rate of `86.91%` and a plate miss rate of `13.09%`. At image level, `331` of `386` test images had all annotated plates nearly fully covered by blur regions, giving an image privacy pass rate of `85.75%`.

The secondary `80%` operational view is slightly higher: `456` of `512` plates protected, `89.06%` protection rate, and `87.82%` image privacy pass rate.

The `15%` padding produced the highest privacy protection rate among the tested values. However, larger padding also removes more surrounding visual context. Therefore, `15%` was selected as a privacy-first setting, and the final value should be reviewed against the operational need to preserve vehicle details. This means the system is useful as an automated privacy layer, but it should still be improved before high-risk production deployment where every visible plate must be hidden.

Step 15 - Inference Speed Benchmark

Goal: Avoid claiming real-time performance without measuring speed.

The model was benchmarked locally using the downloaded `best.pt` checkpoint, input size `960`, confidence threshold `0.25`, 5 warm-up runs, and 50 measured runs. The benchmark measures model prediction latency, blur post-processing latency, end-to-end latency, and approximate FPS on the available local CPU environment.

Benchmark Environment

- Processor: `13th Gen Intel(R) Core(TM) i7-13700H`
- RAM: approximately `15.73 GB`
- Operating System: `Windows 11`

- Python Version: `3.12.10`
- Torch Version: `2.13.0+cpu`

Compact end-to-end timing logic used for the measured inference-and-blur loop:

```
start = time.perf_counter()

for _ in range(50):
    result = model.predict(
        image,
        imgsz=960,
        conf=0.25,
        device="cpu",
        verbose=False,
    )[0]

    boxes = result.boxes.xyxy.cpu().numpy()
    blur_detected_regions(image, boxes)

elapsed = time.perf_counter() - start
average_end_to_end_ms = elapsed / 50 * 1000
fps = 1000 / average_end_to_end_ms

BENCHMARK_PATH = PROJECT_DIR / "outputs" / "model_outputs" /
"benchmark" / "inference_benchmark_summary.json"

if not BENCHMARK_PATH.exists():
    raise FileNotFoundError("The precomputed benchmark artifacts are
missing. Recompute them before final PDF export.")

benchmark_summary = pd.read_json(BENCHMARK_PATH, typ="series")
benchmark_df = pd.DataFrame({
    "Item": [
        "Hardware", "Device", "Model", "Input size", "Benchmark runs",
        "Average preprocess time", "Average inference time", "Average
postprocess time",
        "Average model predict latency", "Average blur processing
time",
        "Average end-to-end blur latency", "Approximate end-to-end
FPS",
    ],
    "Value": [
        "13th Gen Intel(R) Core(TM) i7-13700H",
benchmark_summary["device"], benchmark_summary["model"],
        benchmark_summary["input_size"],
int(benchmark_summary["benchmark_runs"]),
        f"{benchmark_summary['average_preprocess_ms']:.2f} ms/frame",
        f"{benchmark_summary['average_inference_ms']:.2f} ms/frame",
        f"{benchmark_summary['average_postprocess_ms']:.2f} ms/frame",
        f"{benchmark_summary['average_model_predict_latency_ms']:.2f}
```

```
ms/frame",
        f"{benchmark_summary['average_blur_processing_ms']:.2f}
ms/frame",

f"{benchmark_summary['average_end_to_end_blur_latency_ms']:.2f}
ms/frame",
        f"{benchmark_summary['approx_fps_end_to_end']:.2f} FPS",
    ],
})
display(benchmark_df)
```

```
                                  Item  \
0                             Hardware
1                               Device
2                                Model
3                           Input size
4                       Benchmark runs
5             Average preprocess time
6              Average inference time
7            Average postprocess time
8       Average model predict latency
9        Average blur processing time
10  Average end-to-end blur latency
11        Approximate end-to-end FPS

                                                  Value
0   13th Gen Intel(R) Core(TM) i7-13700H
1                                                   cpu
2                                        YOLOv8s best.pt
3                                                   960
4                                                    50
5                                          12.06 ms/frame
6                                         543.41 ms/frame
7                                           2.20 ms/frame
8                                         558.27 ms/frame
9                                          19.10 ms/frame
10                                        577.37 ms/frame
11                                               1.73 FPS
```

Conclusion - Step 15

The measured local CPU benchmark produced an average end-to-end detection-and-blur latency of `577.37 ms/frame`, or approximately `1.73 FPS`. Therefore, this report should not claim that the local CPU pipeline performs real-time video processing.

The correct business wording is: the system is **designed for real-time or near-real-time processing**, but real-time deployment depends on the target hardware. A GPU or optimized inference runtime would be needed for higher-FPS video streams.

GPU latency was not benchmarked in this study because the deployment benchmark was performed locally on CPU.

Step 16 - Model Variant Comparison Limitation

Goal: Be transparent about the YOLOv8s model choice.

A stronger experiment would train `YOLOv8n` and `YOLOv8s` for the same number of epochs, with the same `imgsz`, data split, augmentation settings, and confidence policy. That would allow a direct comparison of recall, mAP50-95, latency, and FPS.

In this project, only the `YOLOv8s` checkpoint was fully trained and evaluated. Therefore, the final report should not claim that `YOLOv8s` experimentally outperformed `YOLOv8n`. Instead, `YOLOv8s` is presented as a justified model selection based on license-plate small-object characteristics and the measured final performance of the trained checkpoint.

```
model_variant_comparison = pd.DataFrame({
    "Model": ["YOLOv8n", "YOLOv8s"],
    "Training status in this project": ["Not trained", "Trained"],
    "Image size": [960, 960],
    "Final test recall": ["Not measured", "0.861"],
    "Final test mAP50-95": ["Not measured", "0.670"],
    "End-to-end CPU latency": ["Not measured", "577.37 ms/frame"],
    "Approximate CPU FPS": ["Not measured", "1.73"],
    "Use in final submission": [
        "Recommended future baseline experiment",
        "Final selected model for this case study",
    ],
})

model_variant_comparison
```

```
     Model Training status in this project  Image size Final test
recall  \
0  YOLOv8n                      Not trained         960      Not
measured
1  YOLOv8s                          Trained         960
0.861

  Final test mAP50-95 End-to-end CPU latency Approximate CPU FPS  \
0        Not measured           Not measured        Not measured
1               0.670        577.37 ms/frame                1.73

                    Use in final submission
0    Recommended future baseline experiment
1  Final selected model for this case study
```

Conclusion - Step 16

The correct conclusion is not "YOLOv8s is experimentally better than YOLOv8n." The correct conclusion is: `YOLOv8s` was selected because license plates are small, detail-sensitive objects and the trained `YOLOv8s` checkpoint achieved strong final test and privacy results. A future controlled `YOLOv8n` baseline would make the model-selection argument even stronger.

Step 17 - Final Business Answers and Recommendations

Business Questions Answered

How are YOLO coordinates converted to OpenCV pixel coordinates?
YOLO labels store `center_x`, `center_y`, `width`, and `height` as normalized values. These are multiplied by image width and height, then converted into `(x1, y1, x2, y2)` corner coordinates for OpenCV drawing and region slicing.

Why overlay bounding boxes before training?
Overlaying labels on raw images is the most direct annotation audit. It confirms that the label files match the images, the coordinate conversion is correct, and the boxes actually surround license plates before the model learns from them.

How did annotation quality impact blurring accuracy?
Blurring accuracy depends on detection-box quality. If annotations are shifted, too loose, or missing, the trained model may leave readable plate characters visible or blur unnecessary vehicle regions.

How were images preprocessed without losing plate details?
The project used YOLOv8 letterbox resizing with `imgsz=960`, preserving aspect ratio while keeping enough resolution for small plate characters and edges. Heavy manual resizing, cropping, or smoothing was avoided.

Why YOLOv8 instead of a two-stage detector?
YOLOv8 is a one-stage detector, so it is suitable for real-time-oriented privacy filtering when deployed on sufficiently fast hardware. The tested local CPU implementation achieved only `1.73 FPS`, so live video deployment would require GPU, edge-hardware, or inference-runtime optimization. A two-stage detector can be accurate but is usually heavier and slower for surveillance workflows.

Which YOLOv8 variant was selected?
`YOLOv8s` was selected because many license plates are small and detail-sensitive, while EDA also showed mixed aspect ratios and outlier box sizes. The trained checkpoint achieved strong final test metrics. A controlled `YOLOv8n` baseline was not trained in this project, so the report does not claim a measured improvement over `YOLOv8n`.

How did license plate characteristics influence model scale?
License plates are often small and text-heavy, but this dataset also contains mixed aspect ratios and large-box outliers. This supported the selection of `imgsz=960` and `YOLOv8s` as a reasonable balance between small-object detail and computational cost.

How was blur applied only inside the detected box?
Each predicted `(x1, y1, x2, y2)` box was used to slice the image region of interest. The box was expanded with `15%` privacy padding, clipped to the image boundary, blurred, and then pasted back into the original frame.

Final Model Performance

The final reported detection performance is based on the untouched test set, not the validation set. The model achieved Precision `0.916`, Recall `0.861`, mAP50 `0.912`, and mAP50-95 `0.670` on 386 test images containing 512 annotated plate instances. Test performance was higher than validation performance, especially mAP50-95, so this result should be interpreted with the split-distribution limitation noted in Step 12.

The final privacy audit at `conf=0.25` and `15%` prediction-box padding uses `95%` ground-truth coverage as the main privacy threshold. Under this stricter rule, the system protected `445` of `512` plates, with a privacy protection rate of `86.91%`, a plate miss rate of `13.09%`, and an image privacy pass rate of `85.75%`. The secondary `80%` operational threshold produced `89.06%` plate protection and `87.82%` image pass rate.

Final Recommendations

- Use the trained `best.pt` checkpoint for inference and blurring.
- Keep the confidence threshold around `0.25` initially, then tune after reviewing false positives and false negatives.
- Prioritize recall, 95% ground-truth coverage, and image-level privacy pass rate because missed or partially exposed plates create direct privacy leakage.
- Add more low-light, angled, motion-blurred, and distant plate examples if future data collection is possible.
- For real-time deployment, test FPS on the target camera hardware; the local CPU benchmark here is suitable for still-image workflows but not real-time video.
- Deploy with TensorRT or ONNX Runtime in production to improve inference latency.
- For future model selection, train `YOLOv8n` and `YOLOv8s` under identical settings to quantify the recall-versus-latency trade-off.
- Evaluate the trained model on an independent public license plate dataset to measure cross-dataset generalization.

Final Conclusion

This project developed an automated YOLOv8-based workflow for detecting and obscuring vehicle license plates while preserving the usefulness of the surrounding traffic scene.

The final YOLOv8s model was evaluated on an untouched test set containing 386 images and 512 annotated license plates. It achieved a precision of `0.916`, recall of `0.861`, mAP50 of `0.912`, and mAP50-95 of `0.670`.

Privacy performance was evaluated using ground-truth plate coverage rather than standard IoU because moderate over-blurring is acceptable, while leaving part of a plate visible creates privacy risk. Under the primary `95%` ground-truth coverage rule, the system protected `445` of `512` plates, producing a privacy protection rate of `86.91%`, a plate miss rate of `13.09%`, and an

image-level privacy pass rate of `85.75%`. Under the secondary `80%` operational rule, the plate protection rate was `89.06%`.

The system therefore provides a useful automated privacy layer, but it does not yet provide a strict privacy guarantee. Remaining failure cases include completely missed plates, additional plates missed in multi-vehicle scenes, very small or angled plates, and occasional false-positive blur regions.

The tested CPU implementation achieved approximately `1.73` frames per second. Consequently, GPU acceleration, model optimization, threshold tuning, and additional difficult-condition training data would be required before live real-time deployment.